logic, providing strong guarantees of policy integrity and preventing subtle but dangerous authorization bugs.

7.4.4 What Cedar doesn’t include

Cedar intentionally omits certain features found in other policy or programming languages, such as regular expressions, floating-point arithmetic, string concatenation, loops, or external calls. These features are excluded not by mistake, but by design: Each one adds risk to performance, complexity, or reasoning ability.

For example, regular expressions can be error-prone and susceptible to denial-of-service attacks like catastrophic backtracking. Similarly, allowing loops or external calls could result in unpredictable execution times and hidden side effects, violating Cedar’s core promise of pure, deterministic evaluation with guaranteed performance limits.

By concentrating on a limited, declarative feature set—such as booleans, strings, sets, entity references, and basic operators—Cedar makes policies easy to read, quick to evaluate, and suitable for formal verification, ensuring they are trustworthy and easy to maintain at scale.

7.4.5 Minimal operators with maximum impact

Cedar provides a deliberately lean set of operators—sufficient to express powerful authorization logic without sacrificing safety, performance, or clarity. Table 7.1 shows an overview of Cedar’s operators.